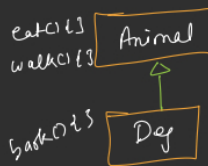


Generics

↳ General purpose class.

Object $\Rightarrow$ class Box $\langle T \rangle$ {
T value;
}

Generics $\Rightarrow$ (Parent-Child Relⁿ)

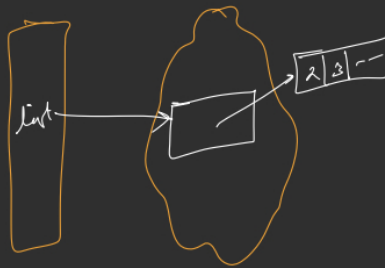


Animal a = new Dog();



List $\langle$ Integer $\rangle$ list = new ArrayList $\langle$ $\rangle$;

list.add(2);
list.add(3);
=

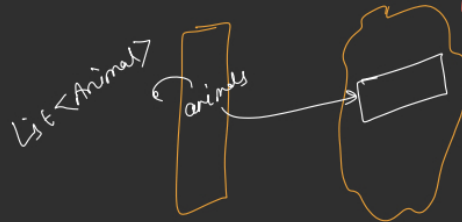


Animal a = new Dog();

List $\langle$ Dog $\rangle$ dogs = new ArrayList $\langle$ $\rangle$;

List $\langle$ Animal $\rangle$ animals = dogs;

(X)



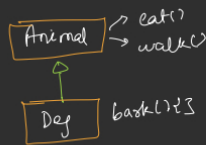
Dog Is-A Animal

List $\langle$ Dog $\rangle$ Is-A List $\langle$ Animal $\rangle$ (X)
Not any Relation.

Generic $\langle A \rangle$ is NOT subtype of Generic $\langle B \rangle$

Generic <A>

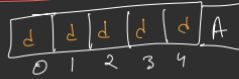
```
List<Dog> dogs = new ArrayList<>();
List<Animal> animals = dogs; (✓)
```



↳ Type Safety will break.

// writing.

```
animals.add(new Animal());
```



```
for (Dog d : animals) {
    d.bark();
}
```

Generics are invariant ✓

g.b A is child of B

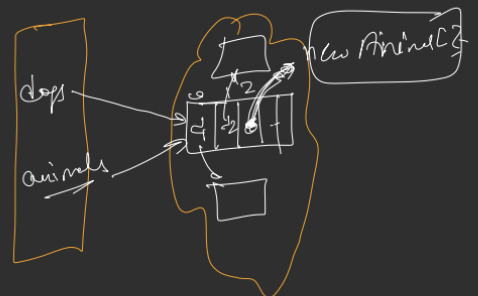
Generic<A> is NOT child of

Generic

```
Dog[] dogs = new Dog[10];
Animal[] animals = dogs; (✓)
```

```
animals[2] = new Animal();
```

↳ Runtime Error.



```
Dog d = new Animal();
```

```
void printAnimals (List<?> a) {
    for (a ---)
    }
}
```

```
printAnimals (dogs);
               (d);

List<Dog> dogs = new ArrayList<>();
```

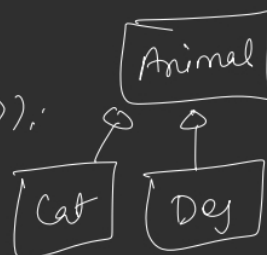
Wild Cards (?)

List<?> list =
new ArrayList<Animal>();
Dog
Cat
1

list of
specific type but
we don't know what?

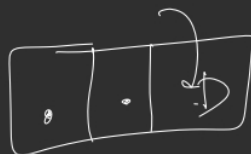
```
void fun( List<?> list) {  
    → list.add("Hello"); X  
    Object obj = list.get(0);  
}
```

```
fun( List<? extends Animal> l) {  
    l.add(new Dog());  
    l.add(new Animal());  
}
```

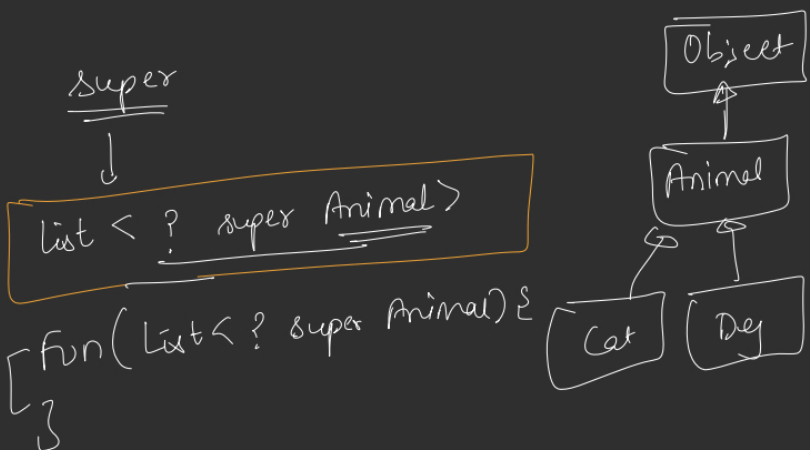


fun(Cats);

fun(Dog);



Covariant ✓



fun (List < Animal >
List < Object >)

// Reading (X)

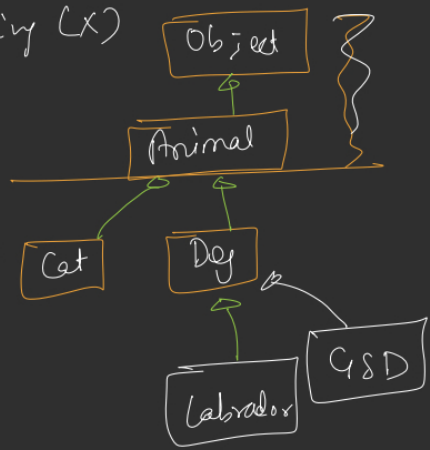
```

fun (List < ? super Animal > l) {
    l.add (new Animal());
    Animal a = l.get(0); (X)
    Object obj = l.get(0) ✓
}

```

fun (animals); // List < Animal >

fun (objects); // List < Object >



List < ? extends T > → Animal

→ Reading (Covariant)

List < ? super T > → Writing (Contravariant)

List < T > → (Invariant)

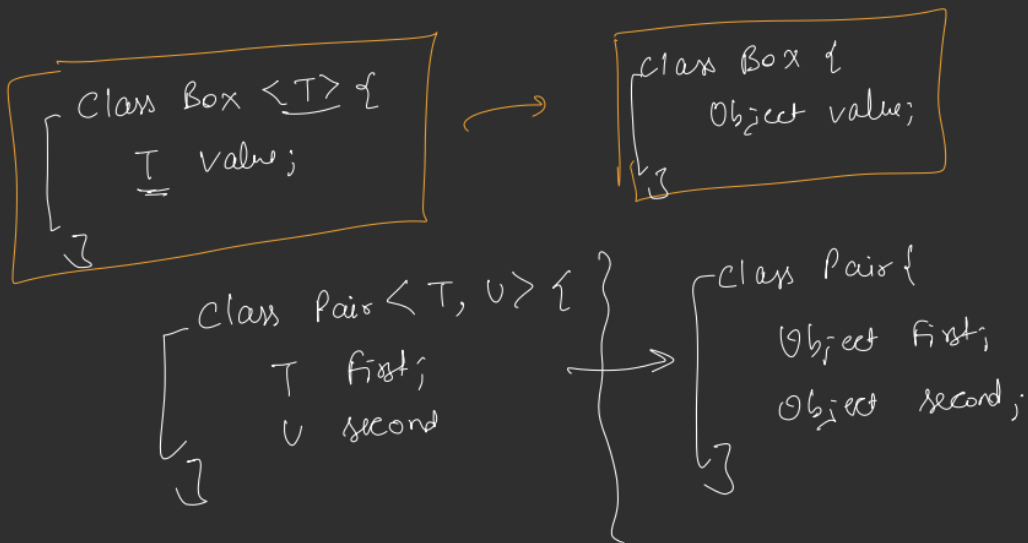
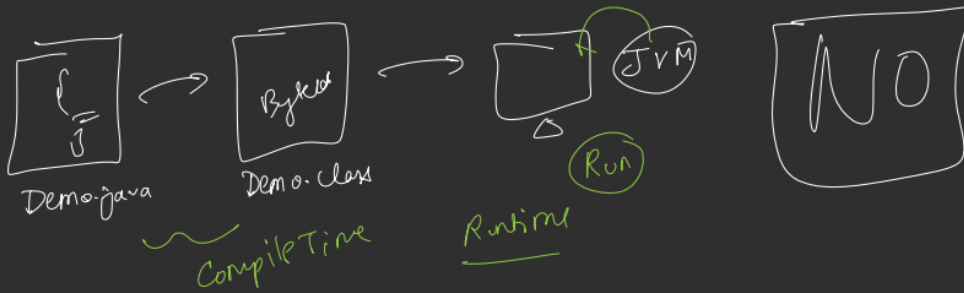
PECS Rule

→ Producer Extends

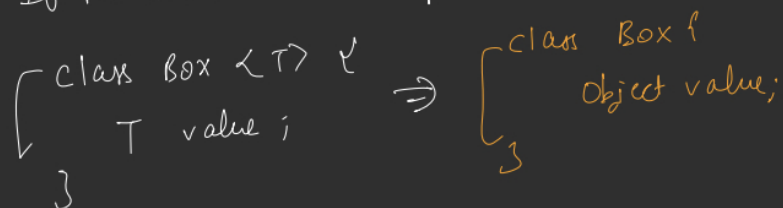
Consumer Super.

Type Erasures

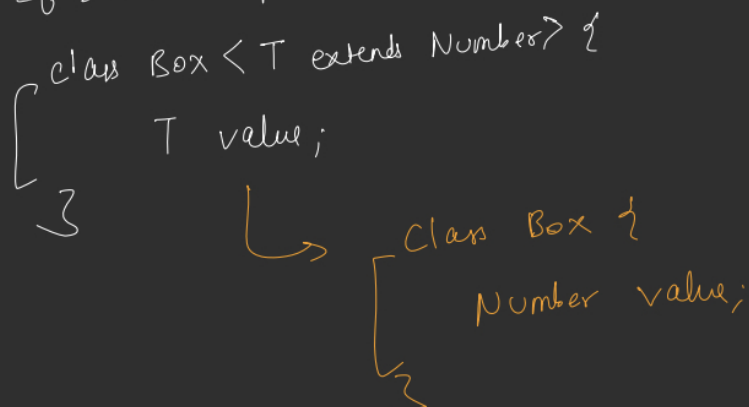
↓
Does JVM Knows Generics?



① If No Bound → Replace Object.



② If bounded replace with bound.



③ Insert Casting automatically.

```
class Box <T> {  
    T value;  
}
```



```
class Box {  
    Object value;  
}
```

```
Box <Integer> b1 =  
    new Box <Integer> (5);  
Integer x = b1.value();
```

```
Box b1 = new Box(5);  
Integer x = b1.value();  
           (Integer)
```

①

```
List <String> l = new ArrayList <> ();
```

```
if ( l instanceof List <String> ) {  
    =  
}
```

```
List l = new ArrayList();
```

```
List <String> → List ✓  
List <Integer> →
```